

게임 기획서

POMOKU 게임 기획서

1. 게임 개요

1.1 게임 타이틀

POMOKU

1.2 장르

전략 카드 보드게임 / 팀 기반 멀티플레이 카드게임

1.3 플랫폼

PC / Steam

1.4 개발 엔진

Unreal Engine 5.6

1.5 게임 설명

POMOKU는 플레이어가 자신의 손패에 있는 카드를 사용해, 보드판 위의 동일한 카드 위치에 칩을 올려놓는 팀 기반 전략 카드게임이다.

플레이어들은 팀 단위로 번갈아 턴을 진행하며, 보드 위에 가로, 세로, 대각선 방향으로 5개의 칩을 연결해 **포목**을 완성해야 한다. 완성된 포목은 카드 조합에 따라 점수를 얻으며, 게임 종료 시 더 높은 점수를 획득한 팀이 승리한다.

1.6 핵심 목표

플레이어는 제한된 손패와 팀의 턴 순서를 활용하여 보드 위에 전략적으로 칩을 배치하고, 더 높은 점수의 포목 족보를 완성하는 것을 목표로 한다.

2. 게임 콘셉트

2.1 핵심 재미

POMOKU의 핵심 재미는 단순히 5개의 칩을 잇는 것이 아니라, **카드 패, 보드 위치, 팀 순서, 족보 점수, 특수 카드 사용 타이밍**이 결합되는 전략성에 있다.

플레이어는 매 턴 다음과 같은 선택을 해야 한다.

- 지금 가진 카드를 바로 사용할 것인가
- 높은 점수의 포목을 노릴 것인가
- 상대의 포목 완성을 방해할 것인가
- 팀원이 이어갈 수 있는 자리를 만들어줄 것인가
- J 또는 조커를 언제 사용할 것인가

즉, POMOKU는 오목의 직관적인 규칙과 카드게임의 확률성, 팀전의 심리전을 결합한 게임이다.

2.2 차별점

기존 오목류 게임과 달리, POMOKU는 원하는 위치에 자유롭게 칩을 놓을 수 없다. 반드시 자신의 손패에 있는 카드와 일치하는 보드 위치에만 칩을 올릴 수 있다.

또한 완성된 5줄이 모두 같은 가치를 가지는 것이 아니라, 완성된 포목 안의 카드 조합에 따라 점수가 달라진다. 따라서 플레이어는 단순 연결뿐 아니라 카드 족보까지 고려해야 한다.

3. 플레이 인원 및 팀 구성

3.1 지원 모드

POMOKU는 기본적으로 팀 기반 멀티플레이를 전제로 한다.

모드	인원	팀 구성
4인 모드	4명	2 vs 2
6인 모드	6명	2 vs 2 vs 2

3.2 팀 플레이 규칙

같은 팀은 승리 목표를 공유하지만, 기본적으로 서로의 손패 정보를 직접 공유할 수 없다.

따라서 팀원 간의 협력은 보드 위에 놓이는 칩의 위치, 턴 순서, 상대 팀의 행동을 통해 간접적으로 이루어진다.

4. 게임 진행 방식

4.1 전체 흐름

1. 플레이어는 로비 화면에서 **방 생성** 또는 **방 찾기**를 통해 방에 입장한다.
2. 정해진 인원이 모두 모이면 방장이 **게임 시작** 버튼을 누른다.
3. 인원 수에 따라 각 플레이어에게 초기 손패가 분배된다.
4. 랜덤으로 시작 순서가 결정된다.
5. 자신의 턴이 되면 손패에서 카드 한 장을 선택한다.
6. 선택한 카드와 일치하는 보드 칸에 칩을 올린다.
7. 칩 배치 후 포목 완성 여부를 검사한다.
8. 포목이 완성되면 족보에 따라 점수를 획득한다.
9. 플레이어는 덱에서 카드 한 장을 뽑는다.
10. 다음 팀 또는 다음 플레이어에게 턴이 넘어간다.
11. 덱의 마지막 카드를 획득한 시점을 기준으로 각 플레이어가 한 턴씩 더 진행한다.
12. 최종 점수가 가장 높은 팀이 승리한다.

4.2 기본 턴 구조

한 턴은 다음 순서로 진행된다.

카드 선택 → 보드 칸 선택 → 칩 배치 → 포목 검사 → 점수 계산 → 카드 드로우 → 턴 종료

4.3 턴 순서

턴은 팀 단위로 번갈아 진행된다.

예를 들어 2vs2 모드에서는 다음과 같은 흐름이 될 수 있다.

A팀 플레이어 1 → B팀 플레이어 1 → A팀 플레이어 2 → B팀 플레이어 2

2vs2vs2 모드에서는 세 팀이 순서대로 번갈아 진행된다.

5. 카드 규칙

5.1 카드 구성

기본 카드 구성은 트럼프 카드를 기반으로 한다.

현재 기획 기준에서는 총 카드 수를 **104장**으로 설정한다.

단, J 카드와 조커 카드의 정확한 포함 방식은 구현 전에 한 번 더 확정할 필요가 있다. 현재 규칙 문서에는 J 4장, 조커 4장이 별도 특수 카드처럼 정의되어 있으므로, 실제 덱 생성 단계에서 카드 수 검증이 필요하다.

5.2 초기 손패

모드	플레이어 수	1인당 초기 손패
2vs2	4명	6장
2vs2vs2	6명	5장

5.3 일반 카드 사용

플레이어는 자신의 손패에서 카드 한 장을 선택하고, 보드판에서 해당 카드와 일치하는 위치에 칩을 올릴 수 있다.

예를 들어 손패에 **하트 7**이 있다면, 보드 위의 하트 7 위치 중 하나에 칩을 놓을 수 있다.

이미 다른 칩이 놓여 있는 칸에는 기본적으로 칩을 놓을 수 없다.

6. 특수 카드 규칙

6.1 J 카드

J 카드는 특수 카드로 사용된다.

J 카드를 사용하면 다음 두 가지 행동 중 하나를 선택할 수 있다.

1. 상대의 칩 하나 제거
2. 원하는 빈 자리에 자신의 칩 하나 배치

단, 이미 완성된 포목에 포함되어 잠긴 칩은 제거할 수 없다.

6.2 조커 카드

조커 카드는 손패를 교체하는 특수 카드다.

사용자는 자기 자신 또는 특정 플레이어 한 명을 지목할 수 있다. 지목된 플레이어의 손패는 전부 덱으로 돌아가고, 새 카드가 다시 지급된다.

조커는 상대의 좋은 패를 방해하거나, 자신의 불리한 패를 리셋하는 전략적 카드로 사용된다.

7. 보드판 규칙

7.1 보드 크기

보드판은 **10×10**, 총 100칸으로 구성된다.

7.2 칸 구성

보드에는 크게 두 종류의 칸이 존재한다.

칸 종류	설명
일반 카드 칸	특정 카드가 배치된 칸
무적 Jari 칸	특수 고정 칸. 칩을 직접 놓는 일반 카드 칸과 다르게 포목 연결을 보조하는 위치

7.3 무적 Jari

무적 Jari는 보드판에 총 4개 배치된다.

무적 Jari는 포목 완성에 활용될 수 있지만, 족보 계산에는 포함되지 않는다.

예를 들어 무적 Jari를 중심으로 주변에 칩 4개가 이어지면 포목이 완성될 수 있다. 하지만 이 경우는 별도 고득점 족보가 아니라 **무적 Jari를 이용한 포목**으로 처리되며, 기본 점수 1점을 획득한다.

8. 보드 생성 규칙

8.1 기본 목표

POMOKU의 보드판은 매 판 동일하지 않다.

보드판은 일정한 규칙에 따라 매 판 새롭게 생성되며, 이를 통해 매치마다 다른 전략 상황을 제공한다.

8.2 생성 순서

보드 생성은 다음 순서로 진행된다.

1. 시드 결정

- 매치 ID를 기반으로 보드 생성 시드를 고정한다.
- 같은 매치에서는 모든 클라이언트가 동일한 보드를 확인할 수 있어야 한다.

2. 대칭 타입 결정

- 점대칭 또는 선대칭 중 하나를 랜덤으로 선택한다.

- 팀 간 공정성을 확보하기 위한 구조다.

3. 무적 Jari 4개 배치

- 각 사분면에 1개씩 배치한다.
- 가장자리는 피한다.
- 무적 Jari끼리는 너무 가까이 배치되지 않도록 한다.

4. 스트레이트 시드 라인 배치

- 3~4개의 스트레이트 가능 라인을 미리 배치한다.
- 가로, 세로, 대각선 방향 중 랜덤으로 선택한다.
- 대칭 구조를 고려하여 공정성을 유지한다.

5. 문양 클러스터 중심점 생성

- 문양별로 특정 지역에 모이는 경향을 만들기 위해 중심점을 생성한다.
- 가능하면 대칭을 고려한다.

6. 나머지 칸 채우기

- 나머지 96칸에 일반 카드를 배치한다.
- 문양은 각 문양이 정확히 24개씩 나오도록 한다.
- 숫자는 주변 연속 숫자와의 연결 가능성을 고려해 가중치를 부여한다.
- 중복 카드의 60~80%는 대칭 위치에 배치한다.

7. 검증

- 각 카드가 정확히 2장씩 존재하는지 확인한다.
- 문양 개수가 24개씩 맞는지 확인한다.
- 시드 라인이 과도하게 겹치지 않는지 확인한다.
- 시작부터 완성에 가까운 5줄이 너무 많지 않은지 확인한다.

8.3 개발 단계별 보드 생성 적용

초기 MVP에서는 고급 보드 생성 규칙을 바로 적용하지 않는다.

처음에는 다음과 같은 단순 보드로 시작한다.

10×10 보드 생성
→ 무적 Jari 4개 배치

→ 나머지 96칸에 카드 랜덤 배치

이후 룰과 멀티플레이가 안정화되면, 시드, 대칭, 클러스터, 검증 로직을 단계적으로 추가한다.

9. 포목 및 점수 규칙

9.1 포목 정의

포목은 자신의 팀 칩 5개가 가로, 세로, 대각선 방향으로 연속해서 이어진 상태를 의미한다.

포목이 완성되면 해당 라인은 잠금 처리된다.

잠금 처리된 칩은 J 카드로 제거할 수 없다.

9.2 점수 목록

족보	점수	조건
스트레이트 포목	5점	5개의 숫자가 연속될 때
플러시 포목	4점	5개의 카드 문양이 모두 같을 때
트리플 포목	3점	같은 숫자 3개를 포함할 때
투페어 포목	2점	같은 숫자 2개 쌍을 포함할 때
노멀 포목	1점	위 조건에 해당하지 않을 때
무적 Jari 포목	1점	무적 Jari를 이용해 완성한 포목

9.3 스트레이트 규칙

스트레이트 포목은 숫자가 연속되는 경우 인정된다.

J는 스트레이트 계산에서 제외된다.

현재 규칙에서는 다음 조합도 스트레이트로 인정한다.

9, 10, Q, K, A

단, 이 규칙은 일반 포커의 스트레이트와 다르기 때문에 UI 또는 튜토리얼에서 명확히 안내할 필요가 있다.

9.4 족보 우선순위

하나의 포목이 여러 족보 조건을 동시에 만족할 경우, 더 높은 점수를 주는 족보를 우선 적용하는 방식이 적합하다.

예를 들어 어떤 포목이 플러시이면서 투페어 조건도 만족한다면, 더 높은 점수인 플러시 포목으로 계산한다.

10. 게임 종료 조건

게임은 덱의 마지막 카드를 누군가 획득한 시점을 기준으로 종료 카운트에 들어간다.

마지막 카드를 뽑은 후 즉시 종료되는 것이 아니라, 각 플레이어 또는 각 팀이 한 턴씩 더 진행한 뒤 게임이 종료된다.

게임 종료 시 각 팀의 총점을 비교한다.

가장 높은 점수를 획득한 팀이 승리한다.

동점 상황에 대한 규칙은 아직 확정되지 않았으므로, 추후 다음 중 하나로 결정할 수 있다.

방식	설명
공동 승리	같은 점수의 팀을 공동 승리 처리
추가 턴	동점 팀끼리 한 턴씩 추가 진행
고득점 족보 우선	더 높은 족보를 많이 만든 팀 승리

초기 구현에서는 **공동 승리** 또는 **추가 턴 없이 동점 처리**가 가장 단순하다.

11. UI 기획

11.1 로비 화면

로비 화면에서는 다음 기능을 제공한다.

- 방 생성
- 방 찾기
- 방 입장
- 플레이어 목록 확인
- 팀 구성 확인
- 준비 상태 확인
- 게임 시작

11.2 게임 HUD

게임 중 HUD에는 다음 정보가 표시되어야 한다.

UI 요소	설명
보드판	10×10 카드 위치와 칩 상태 표시
내 손패	현재 플레이어가 가진 카드 표시
현재 턴	누구의 턴인지 표시
팀 점수	각 팀의 현재 점수 표시
남은 덱 수	남은 카드 수 표시
게임 페이지	대기, 진행, 종료 등 상태 표시

11.3 기본 조작 흐름

플레이어는 다음 순서로 조작한다.

손패 카드 클릭 → 배치 가능한 보드 칸 강조 → 보드 칸 클릭 → 칩 배치

J 카드나 조커 카드처럼 특수 행동이 필요한 경우에는 별도의 선택 UI가 필요하다.

예를 들어 J 카드를 선택하면 다음과 같은 UI가 표시될 수 있다.

[J 사용 방식 선택]
1. 상대 칩 제거
2. 원하는 위치에 칩 배치

12. 멀티플레이 구조

12.1 기본 원칙

POMOKU는 멀티플레이 게임이므로, 서버 권한 구조를 사용해야 한다.

즉, 중요한 판단은 클라이언트가 아니라 서버에서 처리한다.

서버가 담당해야 하는 요소는 다음과 같다.

- 보드 생성
- 덱 셔플
- 카드 분배
- 카드 사용 검증

- 칩 배치 검증
- 포목 검사
- 점수 계산
- 턴 진행
- 게임 종료 판단

12.2 Unreal Framework 구조

클래스	역할
GI_Pomoku	방 생성, 방 찾기, 초대 등 전체 게임 인스턴스 관리
GM_Pomoku	서버 권한 규칙 처리, 턴 처리, 점수 계산
GS_Pomoku	공개 게임 상태 복제, 보드 상태, 점수, 턴 정보 저장
PS_Pomoku	팀 ID, 준비 상태 등 공개 플레이어 정보 저장
PC_Pomoku	플레이어 입력 처리, 서버 RPC 요청, 개인 손패 관리
WBP_Board	보드 UI
WBP_Hand	손패 UI
WBP_HUD	점수, 턴, 덱 수 표시

12.3 복제 대상

모든 플레이어에게 공개되어야 하는 정보는 `GameState` 를 통해 복제한다.

공개 정보 예시는 다음과 같다.

- 보드의 칩 소유 팀
- 잠금 상태
- 무적 Jari 위치
- 현재 턴
- 팀 점수
- 남은 덱 수
- 게임 진행 상태

반대로 손패 정보는 본인에게만 보여야 한다.

따라서 손패는 전체 복제하지 않고, 각 플레이어의 `PlayerController` 를 통해 소유자에게만 전달하는 구조가 적합하다.

13. 데이터 구조 설계

13.1 카드 데이터

```
USTRUCT(BlueprintType)
struct FCard
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadWrite)
    E_Suit Suit;

    UPROPERTY(BlueprintReadWrite)
    E_Rank Rank;

    UPROPERTY(BlueprintReadWrite)
    bool bIsJoker;
};
```

13.2 보드 칸 데이터

```
USTRUCT(BlueprintType)
struct FBoardCell
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadWrite)
    E_CellType CellType;

    UPROPERTY(BlueprintReadWrite)
    FCard Card;

    UPROPERTY(BlueprintReadWrite)
    E_TeamId ChipOwnerTeam;

    UPROPERTY(BlueprintReadWrite)
```

```
    bool bLocked;  
};
```

13.3 주요 Enum

```
UENUM(BlueprintType)  
enum class E_Suit : uint8  
{  
    Spade,  
    Heart,  
    Diamond,  
    Club  
};
```

```
UENUM(BlueprintType)  
enum class E_Rank : uint8  
{  
    A,  
    Two,  
    Three,  
    Four,  
    Five,  
    Six,  
    Seven,  
    Eight,  
    Nine,  
    Ten,  
    J,  
    Q,  
    K,  
    Joker  
};
```

```
UENUM(BlueprintType)  
enum class E_CellType : uint8  
{  
    Normal,  
    AnchorJ  
};
```

```
};

UENUM(BlueprintType)
enum class E_TeamId : uint8
{
    None,
    TeamA,
    TeamB,
    TeamC
};
```

14. 핵심 함수 설계

14.1 StartGame()

게임 시작 시 호출되는 함수다.

주요 역할은 다음과 같다.

```
보드 생성
→ 덱 생성
→ 덱 셔플
→ 플레이어 손패 분배
→ 팀 및 턴 순서 결정
→ 게임 페이즈를 Playing으로 변경
```

14.2 PlayCard(CardId, CellIndex)

플레이어가 일반 카드를 사용해 칩을 놓을 때 호출된다.

처리 순서는 다음과 같다.

```
카드 소유 여부 확인
→ 선택한 보드 칸이 해당 카드와 일치하는지 확인
→ 칸이 비어 있는지 확인
→ 칩 배치
→ 사용한 카드 제거
→ 포목 검사
→ 점수 계산
→ 카드 드로우
→ 턴 넘김
```

14.3 CheckPomoku(CellIndex, TeamId)

칩을 놓은 위치를 기준으로 포묵이 완성되었는지 검사한다.

검사 방향은 네 가지다.

가로
세로
좌상단-우하단 대각선
우상단-좌하단 대각선

14.4 CalculateScore(PomokuLine)

완성된 포묵의 카드 조합을 확인해 점수를 계산한다.

점수 판정 순서는 높은 점수부터 검사하는 것이 좋다.

스트레이트
→ 풀러시
→ 트리플
→ 투페어
→ 노멀

무적 Jari가 포함된 포묵은 별도로 1점 처리한다.

15. 개발 파이프라인

15.1 0단계: 프로젝트 뼈대 세팅

목표는 룰, 네트워크, UI를 얹기 쉬운 기본 구조를 만드는 것이다.

작업 내용은 다음과 같다.

- Unreal Engine 5.6 프로젝트 생성
- Blueprint 또는 C++ 기반 Blank 프로젝트 구성
- Map_Lobby 생성
- Map_Match 생성
- GI_Pomoku , GM_Pomoku , GS_Pomoku , PS_Pomoku 생성

완료 기준은 다음과 같다.

- 맵을 열었을 때 올바른 GameMode와 GameState가 적용된다.
- 플레이 시 기본 프레임워크 클래스가 정상 생성된다.

15.2 1단계: 로컬 MVP 룰 코어 구현

멀티플레이보다 먼저 로컬에서 게임 규칙이 제대로 돌아가야 한다.

우선 구현할 내용은 다음과 같다.

- 카드 데이터 구조
- 보드 데이터 구조
- 단순 보드 생성
- 덱 셔플
- 손패 분배
- 카드 사용
- 칩 배치
- 포목 검사
- 기본 점수 계산
- 턴 넘김

완료 기준은 다음과 같다.

- 4인 또는 6인 기준 손패 수가 맞다.
- 카드 한 장을 내고 칩을 놓을 수 있다.
- 포목 완성 시 점수가 오른다.
- 턴이 정상적으로 순환한다.

15.3 2단계: UI 구현

로컬 룰이 완성되면 UMG를 통해 실제 플레이 가능한 화면을 만든다.

필수 UI는 다음과 같다.

- WBP_Board
- WBP_Hand
- WBP_HUD

완료 기준은 다음과 같다.

- UI만으로 게임 진행이 가능하다.
- 보드, 손패, 점수, 턴 정보가 일관되게 표시된다.

15.4 3단계: 멀티플레이 리플리케이션

이 단계에서는 서버 권한 구조로 전환한다.

핵심은 다음과 같다.

- 서버가 모든 규칙 판단을 담당한다.
- 클라이언트는 행동 요청만 보낸다.
- 공개 정보는 `GameState` 로 복제한다.
- 손패는 본인에게만 전달한다.

완료 기준은 다음과 같다.

- 여러 창에서 같은 보드 상태가 보인다.
- 상대 손패가 보이지 않는다.
- 네트워크 지연 상황에서도 턴과 점수가 어긋나지 않는다.

15.5 4단계: 규칙 확장

멀티 기반이 안정된 후 특수 카드와 세부 점수 규칙을 붙인다.

추가할 내용은 다음과 같다.

- J 카드 제거 효과
- J 카드 원하는 위치 배치 효과
- 조커 손패 리셋
- 족보 점수 전체 적용
- 종료 조건 적용
- 잠긴 칩 제거 불가 처리

완료 기준은 다음과 같다.

- 모든 족보 점수가 규칙대로 계산된다.
- 특수 카드가 멀티 환경에서도 정상 동작한다.
- 엡지케이스에서 게임 상태가 꼬이지 않는다.

15.6 5단계: 고급 보드 생성 규칙 적용

이 단계에서는 POMOKU만의 차별점인 고급 보드 생성 규칙을 적용한다.

추가할 내용은 다음과 같다.

- 매치 ID 기반 시드 생성
- 점대칭 또는 선대칭 보드 생성
- 무적 Jari 사분면 배치
- 스트레이트 시드 라인 배치
- 문양 클러스터 생성
- 카드 수 및 문양 수 검증
- 실패 시 재생성

완료 기준은 다음과 같다.

- 매 판 다른 보드가 나온다.
- 카드 분포가 규칙을 만족한다.
- 팀 간 지나치게 불공정한 보드가 생성되지 않는다.

15.7 6단계: Steam 연동

마지막 단계에서 Steam 멀티플레이 기능을 붙인다.

구현 내용은 다음과 같다.

- OnlineSubsystemSteam 설정
- 방 만들기
- 방 찾기
- 방 참가
- 친구 초대
- 2vs2 / 2vs2vs2 방 구분
- 패키징 빌드 테스트

완료 기준은 다음과 같다.

- Steam 친구 초대로 방 입장이 가능하다.
- 4인 방과 6인 방이 구분된다.
- 실제 Steam 환경에서 매치가 정상 진행된다.

16. MVP 범위

초기 MVP에서는 모든 기능을 한 번에 구현하지 않는다.

우선 다음 기능만 구현하는 것이 적합하다.

16.1 MVP 포함 기능

- 10×10 보드
- 일반 카드 배치
- 4인 또는 6인 손패 분배
- 카드 사용
- 칩 배치
- 턴 진행
- 노멀 포목 점수
- 간단한 UI
- 로컬 테스트

16.2 MVP 제외 기능

초기 단계에서는 다음 기능을 미룬다.

- Steam 연동
- 고급 보드 생성
- 모든 족보 점수
- J 특수 효과
- 조커 효과
- 랭킹 시스템
- 매치메이킹
- 튜토리얼
- 연출 및 사운드

이유는 간단하다. 먼저 룰 코어가 제대로 동작해야 이후 멀티플레이와 Steam 연동을 안정적으로 붙일 수 있기 때문이다.

17. 예상 개발 우선순위

추천 개발 순서는 다음과 같다.

1. 로컬 룰 코어
2. 보드와 손패 UI
3. 멀티플레이 동기화
4. 특수 카드와 족보 점수
5. 고급 보드 생성
6. Steam 방 생성 및 친구 초대
7. 연출, 사운드, 튜토리얼, 밸런싱

이 순서가 좋은 이유는, POMOKU의 핵심이 그래픽보다 **규칙과 네트워크 동기화**에 있기 때문이다.

특히 카드게임은 상대 손패가 보이면 안 되고, 턴 순서와 점수 계산이 절대 틀어지면 안 된다. 따라서 멀티플레이를 붙이기 전에 로컬에서 규칙을 완전히 검증하는 것이 중요하다.

18. 기획상 추가 확정이 필요한 부분

현재 기획을 실제 개발 문서로 확정하기 전에 다음 항목은 한 번 더 정리하는 것이 좋다.

항목	이유
총 카드 수	J와 조커 포함 방식이 명확히 정리되어야 함
J 카드 수	현재 J가 몇 장인지, 어떤 J만 특수카드인지 확정 필요
조커 포함 여부	총 104장 안에 조커가 포함되는지 별도인지 확인 필요
동점 처리	공동 승리, 추가 턴, 족보 우선 중 선택 필요
팀원 간 정보 공유	완전 금지인지, 이모트/핑 정도는 허용할지 결정 필요
무적 Jari 판정	정확히 어떤 형태로 주변 4칩을 인정할지 구체화 필요
포목 중복 점수	하나의 칩이 여러 포목에 동시에 포함될 수 있는지 결정 필요
완성 포목 잠금 범위	포목 전체 칩만 잠그는지, 교차 포목도 허용하는지 결정 필요

19. 한 줄 요약

POMOKU는 카드 패에 따라 보드 위의 위치를 선택하고, 팀원과 함께 5개의 칩을 연결해 족보 점수를 획득하는 **Steam 기반 팀 전략 카드 보드게임**이다.

게임의 핵심은 다음 세 가지다.

- 카드 운
- + 보드 위치 전략

따라서 개발에서도 가장 먼저 완성해야 할 것은 그래픽이나 Steam 기능이 아니라, **카드 사용 → 칩 배치 → 포목 검사 → 점수 계산 → 턴 진행**으로 이어지는 룰 코어다.